

WHAT IS HOOKS IN REACT

SWIPE TO KNOW

Shantilal Kachhawa
@shantilal

01

USESTATE & USEREDUCER

State is mutable and props are immutable, It's help of state we can rendering dynamic changes of component. The useState Hook lets you keep track of data in your components.

Think of useReducer as a more powerful version of useState, perfect for when your component's state gets complex. It works well for states that have multiple parts or when the new state depends on the old one.

02

USEEFFECT & USELAYOUTEFFECT

`useEffect`:

Runs asynchronously after the DOM has been painted. It is suitable for tasks like data fetching, subscriptions, or logging.

`useLayoutEffect`:

Runs synchronously after DOM mutations but before the browser paints. Use it for tasks like measuring DOM elements or synchronizing the UI with the DOM.

03

REACT.MEMO & USEMEMO

React.memo is a Higher-Order Component (HOC) used to memoize a React component — it prevents unnecessary re-rendering if the props haven't changed.

Calculate the prethe data Use memo to return the previous computed value instead of recalculate value again and again with the same input.

Use case: Optimizing functional components that receive the same props but still re-render due to parent re-rendering.

useMemo is a React Hook that memoizes the result of a function — used to avoid expensive calculations unless dependencies change.

Use case: Optimizing heavy functions or computed values during rendering, work with only pure function

04

USECALLBACK

useCallback is a React Hook that memoizes a function — it returns the same function instance unless its dependencies change.

use callback is to avoid creating a new instance until react created a new instance that why render child component

Referential Equality.

05

USEFORMSTATUS

The `useFormStatus` hook gives information about the form's status while submitting the form. This hook is part of the React DOM, so import it from `react-dom`.

You can use any number of forms inside the parent `<form>` element and get the form state with `useFormStatus` hook. So, you do not need to implement the same logic every time. This improves reusability in the code.

06

USEACTIONSTATE

```
const [state, formAction] = useActionState(submitData, initialState);
```

- `submitData`, which is the function that is called when form is submitted. This function should take two parameters: the current state and a `FormData` object.
- `initialState`, which is the initial value of the state when the form is not submitted.
- `state`, which is the current state that is being rendered in the component. This state is equal to the initial state.
- `formAction`, which is the new action that you can pass to the `action` prop of your form element. This executes the action that you passed with the current state and returns a new, updated state.

07

USEOPTIMISTIC

```
const [optimisticState, setOptimisticState] =  
  useOptimistic(actualState, updateFn);
```

- `actualState`, which is the value of the optimistic state when no action is pending.
- `updateFn` (optional), which is a function that takes the actual state and the value passed to the `setOptimisticState` method and calculates the `optimisticState`. If this parameter is not specified, the optimistic state is equal to the new value.
- `optimisticState`, which is the optimistic (temporary) value shown while the action is pending.
- `setOptimisticState`, which is a function that sets the optimistic state to a new value.

08

USEREF & FORWARDREF

The `useRef` hook in React is used to create a mutable object that persists across renders. It can be used to access and manipulate DOM elements directly, store mutable values that do not cause re-renders when updated, and keep a reference to a value without triggering a re-render. For example, you can use `useRef` to focus an input element

`ForwardRef` Allows parent to pass a ref to child's DOM node.

Useful in component libraries or reusable components.

09

USEID & FORWARDREF

useld is a React Hook for generating unique IDs that can be passed to accessibility attributes.

Usage-

- Generating unique IDs for accessibility attributes
- Generating IDs for several related elements
- Specifying a shared prefix for all generated IDs
- Using the same ID prefix on the client and the server

WANT MORE TIPS?

**FOLLOW US AND SHARE
THIS POST WITH OTHERS
WHO MIGHT FIND IT
HELPFUL!**

Shantilal Kachhawa

@shantilal

